SKILLING-ML/DL

II-II CSE, CSE(AI&ML)

AY: 2025-2026

WEEK 2 – Supervised Learning: Regression & Classification

SL No	Week	Session-1 (3 Hours)	Detailed Subtopics	Sessio-2 (3 Hours)	Detailed Subtopics
2	Week 2	Regression & Classification Models	Simple and multiple linear regression, Cost function (MSE), Gradient descent optimization, Logistic regression, Decision boundaries, k-Nearest Neighbors, Bias-variance trade-off	Model Training Validation, & Feature Engineering	Train-test split, Cross-validation techniques, Feature scaling, Model comparison strategies, Interpretation of results

SESSION2:

PART A: THEORY – Model Training, Validation & Feature Engineering (1hr)

Contents

1. Train–Test Split
2. Cross Validation
3. Feature Engineering
4. Feature Scaling
5. Standardization
6. Normalization
7. Metrics
8. Interpretation

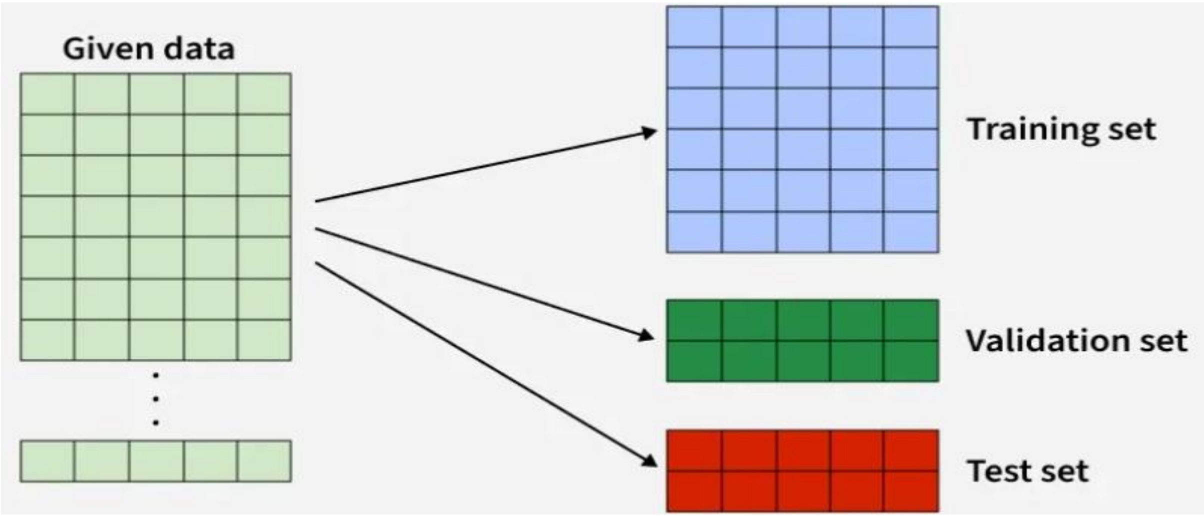
1. Train–Test Split

What is Train–Test Split?

In machine learning, we never test a model on the same data used for training, because that would give false confidence.

So, we split the dataset into:

- Training data → used to learn patterns
- Testing data → used to evaluate performance



Example

House price dataset with 1000 records:

- 800 → Training
- 200 → Testing

The model learns from 800 houses and is tested on 200 unseen houses.

Real-Life Analogy
Studying from a question bank → **training**
Writing a final exam with new questions → **testing**

Dataset	Purpose	Usage Example
Training Set	Used to train the model. The model learns patterns from this data.	If predicting house prices, model learns relationship between features (size, location) and price.
Validation Set	Used to tune hyperparameters and prevent overfitting. Helps select the best model.	Choosing the number of layers, learning rate, or regularization strength.
Test Set	Used to evaluate final model performance on unseen data. Simulates real-world performance.	After training and validation, check model accuracy on completely new houses.

Why Train Test Split is Important

- Detects overfitting
- Measures generalization ability
- Simulates real-world prediction

2. Cross-Validation (CV)

Problem with Single Train–Test Split

Model performance may depend on how data is split.

One bad split = misleading accuracy.

Solution: Cross-Validation

k-Fold Cross Validation

1. Split data into **k equal parts (folds)**
2. Train on k–1 folds
3. Test on 1 fold

4. Repeat k times
5. Average the results

Example (5-Fold CV)

Dataset: 100 samples

Fold	Used for Testing	Used for Training
1	Fold 1	Fold 2–5
2	Fold 2	Fold 1,3,4,5
3	Fold 3	Fold 1,2,4,5
4	Fold 4	Fold 1–3,5
5	Fold 5	Fold 1–4

Final accuracy = average of 5 accuracies

When to Use CV?

- Small datasets
- Model comparison

Real-Life Analogy

Interviewing a candidate:

One interviewer's opinion may be biased

Multiple interviewers → more reliable judgment

3. Feature Scaling

Feature scaling means bringing all features to a similar scale.

Why Feature Scaling is Needed?

Example (Before Scaling)

Feature	Range
Area (sqft)	500 – 5000

Feature	Range
Bedrooms	1 – 5
Age	1 – 30

Algorithms may think:

"Area is more important just because values are bigger"

This is **wrong**.

Algorithms that NEED Scaling

- Linear Regression (Gradient Descent)
- Logistic Regression
- k-NN
- SVM
- Neural Networks

4. Standardization

What is Standardization?

Transforms data to:

- Mean = **0**
- Standard Deviation = **1**

Formula

$$z = \frac{x - \mu}{\sigma}$$

Where:

- x= original value
- μ= mean

- σ= standard deviation

Example

Feature: Exam scores

Values: 50, 60, 70, 80, 90

Mean = 70

Std Dev = 14.14

For x = 90:

$$z = \frac{90 - 70}{14.14} = 1.41$$

After Standardization

- Data centered around 0
- Negative values allowed
- No fixed range

When to Use Standardization?

- When data follows Gaussian distribution
- Algorithms using gradients or distance
- Linear & Logistic Regression

Real-Life Analogy

Converting marks from different universities to **z-scores** for fair ranking.

5. Normalization

What is Normalization?

Rescales data to a **fixed range**, usually **0 to 1**

Formula (Min–Max Scaling)

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Example

Feature: House Area

Min = 500

Max = 5000

For x = 2500:

$$x_{norm} = \frac{2500 - 500}{5000 - 500} = 0.44$$

After Normalization

- All values lie between 0 and 1
- Preserves original distribution shape

When to Use Normalization?

- k-NN
- Neural Networks
- When bounded values are needed

Real-Life Analogy

Converting marks to **percentage**

6. Standardization vs Normalization

Aspect	Standardization	Normalization
Range	No fixed range	0 to 1
Mean	0	Not fixed
Std Dev	1	Not fixed
Sensitive to outliers	Less	More
Used in	LR, Logistic, SVM	k-NN, NN

7. Feature Engineering

Feature Engineering = Creating better input features to improve model performance.

“Better features beat better algorithms.”

Types of Feature Engineering

7.1 Feature Creation

Example

Original:

- Date of purchase

Create:

- Day
- Month
- Year
- Is_Weekend

7.2 Feature Transformation

Example

Salary values:

- 10,000
- 10,00,000

Apply:

- Log transformation to reduce skew

7.3 Feature Encoding

Categorical → Numerical

City	Encoded
Hyderabad	0
Chennai	1
Bangalore	2

OR One-Hot Encoding

7.4 Feature Selection

Removing irrelevant features.

Example

Predicting house price:

- Area ✔
- Location ✔
- Owner name ❌

7.5 Feature Scaling

- Standardization
- Normalization

Real-Life Analogy

Cooking:

Feature engineering = chopping, cleaning, marinating

Algorithm = cooking method

Bad ingredients → bad dish

8. Model Evaluation Metrics

Regression

- MSE – average squared error
- RMSE – error in original units
- R^2 – goodness of fit

Classification

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Confusion Matrix

A Confusion Matrix is a table used to evaluate the performance of a classification model. It shows the number of correct and incorrect predictions categorized by actual vs predicted classes.

For **binary classification**, the matrix looks like this:

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Definitions:

- **True Positive (TP):** Model predicted positive, and it is actually positive.
- **False Positive (FP):** Model predicted positive, but it is actually negative. (Type I error)
- **True Negative (TN):** Model predicted negative, and it is actually negative.

- **False Negative (FN):** Model predicted negative, but it is actually positive. (Type II error)

Metrics Derived from Confusion Matrix

1. **Accuracy**

   Measures overall correctness of the model.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

When to use:

   - Good when classes are **balanced** (similar number of positive and negative examples).

     Example:

   - Disease detection: 950 healthy, 50 sick. Model predicts all healthy → Accuracy = 95% (misleading!)

2. **Precision** (Positive Predictive Value)

   Measures correctness of positive predictions.

$$\text{Precision} = \frac{TP}{TP + FP}$$

When to use:

   - When false positives are costly.

     Example:

   - Email spam detection: Predicting non-spam as spam can delete important emails → want high precision.

3. **Recall (Sensitivity / True Positive Rate)**

   Measures ability to identify actual positives.

$$\text{Recall} = \frac{TP}{TP + FN}$$

When to use:

  - When false negatives are costly.
    Example:
  - Cancer detection: Missing a sick patient → dangerous → want high recall.

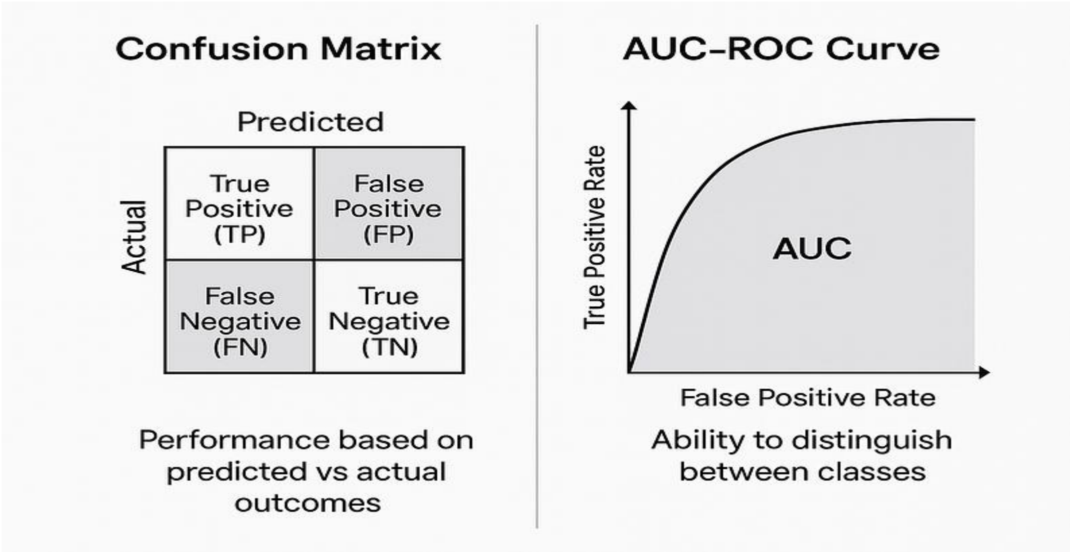
4. **F1 Score**
   Harmonic mean of precision and recall. Useful when you want a balance between precision and recall.

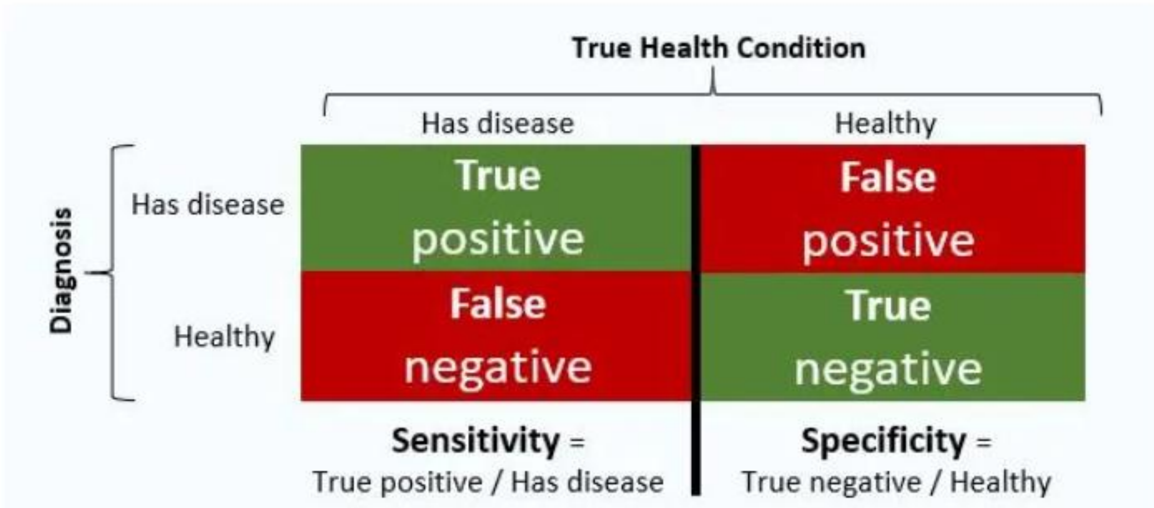
$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

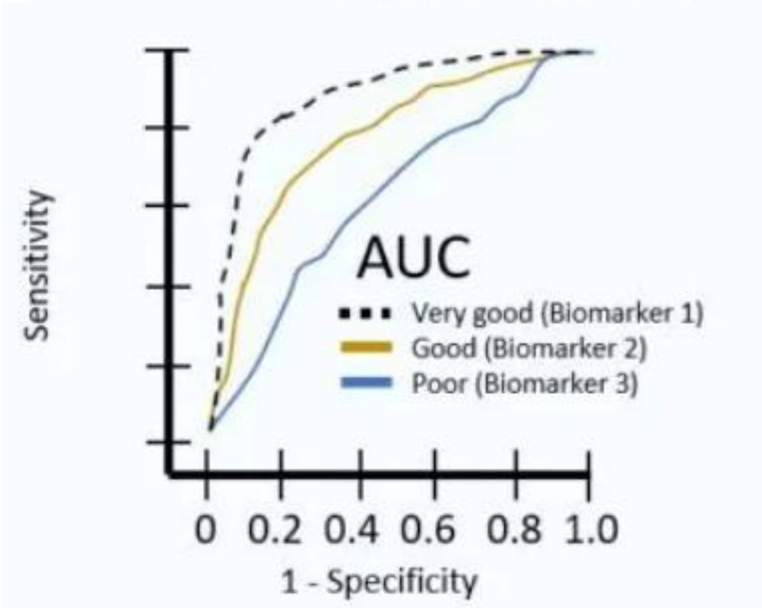
5. **ROC Curve (Receiver Operating Characteristic)**
   - Plots True Positive Rate (Recall) vs False Positive Rate (FPR = FP / (FP+TN)) at different thresholds.
   - Helps visualize trade-offs between sensitivity and specificity.

6. **AUC (Area Under the Curve)**
   - Measures overall model performance across thresholds.
   - Value between 0 and 1; higher is better.
     Example:
   - AUC = 0.95 → model can distinguish positives from negatives 95% of the time.

The Confusion Matrix offers a detailed snapshot of a model's prediction accuracy by comparing actual vs. predicted values, while the AUC-ROC Curve evaluates the model's overall ability to distinguish between classes.







The Receiver Operator Characteristic (ROC) curve is an evaluation metric for binary classification problems. It is a probability curve that plots the TPR against FPR at various threshold values and essentially separates the 'signal' from the 'noise'. The Area Under the Curve (AUC) is the measure of the ability of a classifier to distinguish between classes and is used as a summary of the ROC curve.

AUC-ROC curve is a performance measurement for the *classification problems* at *various threshold* settings. ROC is a probability curve, and AUC represents the degree or measure of separability. It tells how much model is capable of distinguishing between classes. Higher the AUC, better the model is at *predicting 0s as 0s and 1s as 1s*. By analogy, Higher the AUC, better the model is at distinguishing between *patients with the disease and no disease*.

The ROC curve is plotted with TPR against the FPR where TPR is on the y-axis and FPR is on the x-axis.

Scenario 1: Fraud Detection

- Class imbalance: 990 legitimate transactions, 10 fraudulent.
- Model predicts 5 fraudulent correctly (TP=5), predicts 5 legitimate as fraudulent (FP=5), misses 5 frauds (FN=5).

	Predicted Fraud	**Predicted Legit**
Actual Fraud	5 (TP)	5 (FN)
Actual Legit	5 (FP)	985 (TN)

- Accuracy: (5+985)/1000 = 99% → looks great, but model missed half the frauds.
- Precision: 5/(5+5) = 50% → half of predicted frauds are wrong.
- Recall: 5/(5+5) = 50% → only detected half of actual frauds.
- F1 Score: 50% → balances precision and recall.

Use Case:

- **Recall is more important here** (we don't want to miss frauds).
- AUC-ROC curve will help check overall model discrimination ability.

Scenario 2: Spam Detection

- False positives (marking real emails as spam) are annoying.
- **Precision is more important** (we want flagged emails to truly be spam).

Summary:

Metric	**Focus**	**When Important**
Accuracy	Overall correctness	Balanced classes
Precision	Correctness of positive preds	Costly false positives
Recall	Capturing actual positives	Costly false negatives
F1 Score	Balance of precision & recall	When both FP & FN matter
AUC-ROC	Threshold-independent ability	Comparing classifiers, imbalanced data

9. Interpretation of Results

Overfitting

- High training accuracy
- Low testing accuracy

Underfitting

- Poor performance everywhere

Bias–Variance Trade-Off

Bias	**Variance**
Too simple	Too complex

Bias	Variance
Underfit	Overfit

Goal: Balance both.

1. Question: A hospital builds a model to detect a rare disease. The model predicts all patients as healthy. What is misleading about the accuracy metric here?

Answer:

- Accuracy might appear high because most patients are healthy (class imbalance).
- **Lesson:** Accuracy is unreliable when classes are imbalanced; use **Recall, Precision, F1** instead.

2. Question: You're developing a spam filter. Predicting legitimate emails as spam annoys users. Which metric should you prioritize and why?

Answer:

- **Precision** should be prioritized.
- High precision ensures flagged emails are truly spam, reducing false positives.

3. Question: A fraud detection system catches most frauds but sometimes flags legitimate transactions as fraud. Which metric explains this trade-off?

Answer:

- **Recall** measures catching actual frauds (TP/(TP+FN)).
- **Precision** measures correctness of flagged frauds (TP/(TP+FP)).
- **F1 Score** balances both.

4. Question: You have an email classifier. The ROC curve is close to the diagonal line. What does it indicate?

Answer:

- The model is performing **like random guessing**.
- AUC ≈ 0.5 → poor discrimination between classes.

5. Question: In a COVID-19 test, missing an infected patient is dangerous. Which metric is **critical**, and why?

Answer:

- **Recall (Sensitivity)** is critical.
- Ensures infected patients are detected (minimize false negatives).

6. Question: Your model predicts 40 positive cases, of which 30 are correct. 20 actual positives were missed. Calculate precision, recall, and F1.

Answer:

- **TP = 30, FP = 10, FN = 20**
- Precision = 30/(30+10) = 0.75
- Recall = 30/(30+20) = 0.6
- F1 = 2*(0.75*0.6)/(0.75+0.6) ≈ 0.667

7. Question: Why might a model with 99% accuracy still be considered **bad** for fraud detection?

Answer:

- Fraud is rare → model predicts almost all transactions as legitimate → misses frauds.
- **Metrics like Recall, Precision, and AUC-ROC** are more meaningful.

8. Question: Your classifier has high precision but low recall. What kind of mistakes is it making?

Answer:

- Making **few false positives** (good), but **many false negatives** (misses actual positives).
- Example: Spam filter that rarely flags emails → misses many spam emails.

9. Question : A bank's loan approval model has ROC-AUC = 0.95. What does this tell you?

Answer:

- The model can distinguish approved vs rejected applicants **95% of the time** across all thresholds.
- Strong classifier performance regardless of threshold choice.

10. Question : A machine learning model shows the following: TP=50, FP=10, FN=40, TN=900. Which metric would you prioritize for detecting faulty products in a factory and why?

Answer:

- TP=50, FN=40 → model misses many faulty products.
- **Recall** is important to catch as many faulty products as possible.
- Accuracy = (50+900)/1000 = 95% → looks good but **misses many faults**, so not reliable alone